# A Pre-Registered Cost Model for a Small Bit-Exact int8 CNN Accelerator on a RISC-V Microcontroller SoC

Anonymous for review

Abstract—We present mcuaccel, an int8 convolution, depthwise and fully connected accelerator memory-mapped into a PicoRV32 SoC, together with a backend for mcugrad that emits accelerator command descriptors and driver C while keeping every other operator in the original software kernels. The accelerator targets bit-exactness with the TFLite-style integer reference. Its cycle counts are predicted by an analytic port-bound model whose predictions were committed to git (tag m1, 6533b1e) before any datapath RTL existed. At $P = 8$ the frozen model predicts a speedup of 27.0× on mnist_cnn and 363.1× on cifar_mobilenet; these were computed against the software baselines measured before decision D19, an inlining fix after which the baselines are 11,383,383 and 69,169,620 cycles per inference. Measured against the current baselines: 46.8× and 350.0×. A post-hoc rescoring, which substitutes the measured post-D19 fallback-operator cycles into the frozen accelerator-plus-driver prediction, gives 46.7× and 348.0×. Claim C1 is confirmed; C4a is confirmed, although 6 commands exceeded their pre-registered bias (root cause given post hoc below); C3 is refuted: 7,313 LUT4 against a budget of 6,000. All cycle counts are Verilator cycles, not FPGA wall-clock.

## I. Introduction

Quantized CNNs cost tens of millions of cycles per image on microcontroller-class cores. A dedicated int8 accelerator beating a scalar in-order RISC-V core by an order of magnitude or more is well established [1]–[3]; reproducing that is not our contribution. We ask instead how well a simple analytic model, written down before the RTL, predicts the cycle counts of a small memory-bound accelerator, with tolerances fixed in advance. The claims below are pre-registered in docs/SPEC.md; the predictions are frozen at tag m1 (commit 6533b1e, m1_predictions.csv sha256 prefix fd94f39a1a913ee6).

- C1 Speedup at $P = 8$ is at least 10× on both networks, end to end with fallback layers and driver. C1b: at least 100× on cifar_mobilenet.
- C2 Bit-exactness: zero mismatching bytes on 1,000 random layers (C2a), in every intermediate tensor of both networks (C2b), and in final outputs for 100 images per network on the full SoC (C2c).
- C3 At $P = 8$ the accelerator uses at most 6,000 LUT4, 40 MULT18X18D and 12 DP16KD on an ECP5 LFE5U-85F.
- C4 The frozen model predicts every accelerator command within 20 % (C4a, all $P$) and each network's accelerator-plus-driver time within 10 % (C4b).

PicoRV32 → adapter → interconnect → {RAM port A, console, accelerator registers}
fetch (only master) → ctrl (descriptor FSM, loop nest) → align → MAC array ($P$ outputs × 4 lanes) → requant → writer → RAM port B

Fig. 1. System block diagram (schematic; docs/DESIGN.md Sec. 1).

- C5 The saturation width of the $P$ sweep is reported, predicted versus measured; its mechanism was stated in advance.

## II. Background and Related Work

Quantization. The numerical contract is the integer-only scheme of Jacob et al. [4] as implemented in gemmlowp [5] and specified by LiteRT [6]: a saturating rounding doubling high multiply followed by a rounding divide by a power of two. MCU inference. TFLite Micro [7], CMSIS-NN [8] and MCUNet [9] provide software kernels; mcugrad [10] is the compiler whose int8 simulator is our specification. Accelerators. Eyeriss [11], the TPU [12], NVDLA [13], VTA [3], Gemmini [1] and CFU Playground [2] show large speedups over in-order cores; FINN [14] and PULP-NN [15] target FPGA and multicore MCU settings. None of their RTL is used here. Models. Earlier analytic models [16], [17] and the roofline model [18] were fitted alongside their designs; ours was frozen first. The CPU is PicoRV32 [19] on a Lattice ECP5 [20], with the bus protocol of corv32 [21].

## III. Architecture

The CPU reaches a dual-port RAM through an adapter and interconnect; the accelerator is a second bus master on the RAM's other port (a private 32-bit port, one word per cycle, one outstanding request), so no arbiter is needed. The CPU writes a descriptor address and a start bit, then polls a status register (Fig. 1).

Dataflow. The array is output-stationary over one output pixel and a tile of $P$ output channels. In NHWC layout the in-bounds part of a kernel row is contiguous, so each cycle one four-byte activation word meets one weight-buffer row of $4{\cdot}P$ bytes and performs $4{\cdot}P$ multiply-accumulates; every channel count uses one mechanism. Depthwise layers get only 4 useful MACs per cycle. Weights and per-channel parameters are loaded once per

command into local buffers (512 rows). A single requantizer emits one byte per cycle using the specification's SRDHM/RDBPOT rule; accumulators are 32 bit and the backend enforces a no-wrap bound.

Interface. A command is a 18-word descriptor in RAM (geometry, strides, addresses, zero points, clamp). Descriptors chain by pointer and the accelerator writes a cycle stamp into each. Anything outside the conv/depthwise/fully-connected envelope falls back to software.

## IV. Backend and Driver

The backend (backend/mcuaccel_backend) consumes mcugrad's int8 graph, packs weights and parameters, splits layers that exceed buffer limits, builds descriptors, and emits a model.c with the same model_run API; unsupported ops are the identical mcugrad kernels. A pure-Python executor of the descriptor semantics, tested equal to the integer reference, is the oracle for the RTL's request stream. The driver writes the descriptor address, starts the chain, polls, and patches only the fields that depend on the network input and output. Its cost, measured on the walking-skeleton stub, is 71 cycles per call plus 15 per patched field.

## V. Methodology

Pre-registration. Tag m1 precedes every commit that adds a datapath or sequencer. The model is not changed to match the RTL, nor the RTL tuned to the model: the RTL is scored at its first full measurement and later cycle-changing edits are logged with before/after values. Predicted rows are those of m1_predictions.csv with architecture os (17 commands at $P = 8$).

Timing protocol. Inference time is the rdcycle interval around model_run on the SoC, including software layers, descriptor patching, polling and driver cost, over 10 fixed test images and 3 timed runs after a warm-up (decision D17: the simulator is deterministic, so more runs add nothing). We report the median of per-run per-image medians with min and max. The bench window costs 120 cycles in both builds. Cycle counts come from Verilator simulation of the full SoC at a nominal 50 MHz.

Deviations after m1. Two logged changes altered measured cycles; neither changed the frozen predictions or a threshold (values in results/deviations.json, written by experiments/make_deviations.py from the decision text with the after-values checked against results/). D19: identical fallback source compiled to different code in the two builds, because inlining depended on which operators a file held. After pinning (below) the baseline moved from 12,188,028 to 11,383,383 (mnist) and 72,157,425 to 69,169,620 (cifar) cycles, and the accelerated $P = 8$ build from 451,480 to 243,059 and 197,617 to 197,613; measured speedup moved from 27.0× to 46.8× (mnist) and 365.1× to 350.0× (cifar). D20: the RTL's credit return had one register stage too many against the normative timing (Sec. 7.2 of the design); it was made combinational. Smoke runs on the earlier RTL gave a C4b error of -1.0 % (mnist) and +0.4 % (cifar); the scored RTL gives -0.9 % and +0.4 %.

TABLE I
Software baseline on the SoC (cycles per inference).

| Network | median | min | max |
|---|---|---|---|
| mnist_cnn | 11,383,383 | 11,378,852 | 11,387,883 |
| cifar_mobilenet | 69,169,620 | 69,144,241 | 69,186,835 |

Fairness. The baseline is mcugrad's generated model.c with two build-time attribute changes (decision D19): the mg_* kernels are always_inline and each op_* function is noinline, so a fallback operator is compiled from its own text alone; kernel and operator source text is unchanged. The same pinning is applied to the baseline and to the accelerated build, which use the same compiler and flags on the same SoC (accelerator present and idle in the baseline). Fallback-operator identity is checked by comparing per-op profile cycles of the two builds, not by object-code hash. Optional backend passes are off in every scored build.

## VI. Results

Baseline. Table I gives the software baseline. Convolution and fully connected layers account for 98.0 % (mnist) and 99.9 % (cifar) of baseline cycles in the profile build, which bounds the Amdahl ceiling.

C1/C1b: speedup. Table II compares the frozen prediction with measurement. At $P = 8$ the prediction puts the software ops at 95.8 % (mnist) and 31.6 % (cifar) of the accelerated run: pooling and layout ops that stay in software dominate MNIST, whose predicted speedup saturates near 27.2× at the largest $P$. C1 is confirmed; C1b is confirmed. The frozen predictions in the table were computed against the pre-D19 baselines (12,188,028 and 72,157,425 cycles) and pre-D19 fallback-operator cycles, so they are not comparable one to one with the measured columns; the last row is a post-hoc rescoring (frozen accelerator-plus-driver prediction, plus the measured fallback-operator cycles of the accelerated build, plus the 120-cycle bench window): 243,815 and 198,760 cycles against measured 243,059 and 197,613.

C2: bit-exactness. Random layers: 1,000 run, 0 mismatching bytes (C2a confirmed). Intermediate tensors: 10 images, 0 mismatches. Full SoC: 100 images, 0 mismatches against the integer reference and 0 (mnist), [MEASURE: C2c vs baseline firmware cifar mobilenet] (cifar) against the baseline firmware's outputs (C2c [MEASURE: C2c verdict]).

C3: resources. Table III reports the Yosys/nextpnr-ecp5 flow on the LFE5U-85F. The accelerator at $P = 8$ uses 7,313 LUT4, 36 MULT18X18D and 11 DP16KD; C3 is refuted.

TABLE II
Speedup versus $P$: frozen prediction (against pre-D19 baselines) and measurement; last row: post-hoc rescoring at $P = 8$ in the prediction columns.

| | mnist | | cifar | |
|---|---|---|---|---|
| $P$ | pred. | meas. | pred. | meas. |
| 1 | 22.9 | 35.2 | 111.7 | 107.2 |
| 2 | 25.2 | 41.2 | 187.1 | 179.3 |
| 4 | 26.4 | 45.0 | 282.0 | 271.2 |
| 8 | 27.0 | 46.8 | 363.1 | 350.0 |
| 16 | 27.2 | 47.5 | 411.4 | 398.4 |
| 8 post-hoc | 46.7 | 46.8 | 348.0 | 350.0 |

TABLE III
Synthesis (resources from the lowest seed of the newest RTL version of each design; Fmax median over its distinct seeds).

| Design | LUT4 | MULT | BRAM | Fmax MHz | seeds |
|---|---|---|---|---|---|
| PicoRV32 | 2,896 | 4 | 0 | 81.7 | 10 |
| Accel., $P = 8$ | 7,313 | 36 | 11 | 61.5 | 10 |
| SoC, no accel. | 3,392 | 4 | 128 | 59.6 | 3 |
| SoC with accel. | 10,684 | 40 | 139 | 35.8 | 2 |

C4/C5: prediction accuracy. Table V lists predicted command cycles at $P = 8$ against the stamp measurement. Over the commands of the sweep the maximum absolute error is 12.4 %, the median signed error 0.2 %, with 0 tolerance violations out of 108 rows (C4a confirmed). Per network, accelerator plus driver, the signed error is -0.9 % (mnist) and 0.4 % (cifar). Figure 2 overlays the measurement on the frozen prediction: summed STAMP per inference versus $P$, and every scored command against the ±20 % band. The predicted saturation width is 16 (mnist) and 16 (cifar) on the grid, where 16 means not reached, against measured 16 and 16. The model attributes the flattening to the depthwise layers, the single requantizer and the weight load, not to the port.

C4 post-hoc notes. The pre-registered bias list (design Sec. 7.4) gave an over-prediction for a few commands and under 0.5 % for all others. 6 scored rows exceed their registered value by more than 0.5 percentage points (Table IV); all stay inside the tolerance, so C4a is unchanged. Root cause, found after measurement (post hoc, red-team probe on the RTL): when the port term $\lambda(R+W)$ ties the requantizer term, a write of the previous tile is granted in the setup cycle, which is then hidden; the RTL behaves as a maximum over $1 + \lambda R$, $\lambda(R + W)$ and the requantizer term, whereas the model adds the setup cycle to the maximum. The registered bias counted only strictly requantizer-bound pixels. In mnist op 1 at $P = 8$, 546 of 784 setup cycles coincide with a write grant; in cifar op 0 at $P = 16$, 690 of 1,024. The other listed rows were not probed. The model outputs stay as scored (decision D21).

The per-chain cycle counter obeys CYCLES = $\sum$(STAMP+$\lambda$+1)+1 on 275 of 275 chain calls; the frozen

TABLE IV
Commands whose error exceeds the pre-registered bias (cycles; error and registered bias in %).

| net | op | $P$ | pred. | meas. | err. % | reg. % |
|---|---|---|---|---|---|---|
| mnist | 1 | 8 | 7,144 | 6,356 | 12.4 | 3.4 |
| mnist | 1 | 16 | 7,168 | 6,380 | 12.4 | 3.4 |
| mnist | 3 | 16 | 4,580 | 4,515 | 1.4 | 0.5 |
| cifar | 0 | 4 | 50,092 | 49,381 | 1.4 | 0.5 |
| cifar | 0 | 8 | 27,724 | 27,029 | 2.6 | 0.5 |
| cifar | 0 | 16 | 17,648 | 16,616 | 6.2 | 1.9 |

TABLE V
Per-command cycles at $P = 8$ (pred. from tag m1; meas. = median STAMP over the timed images).

| net | op | kind | K | pred. | meas. |
|---|---|---|---|---|---|
| mnist | 1 | conv | 3x3 | 7,144 | 6,356 |
| mnist | 3 | conv | 3x3 | 7,756 | 7,715 |
| mnist | 7 | fc | 1x1 | 3,596 | 3,594 |
| cifar | 0 | conv | 3x3 | 27,724 | 27,029 |
| cifar | 1 | dw | 3x3 | 10,236 | 10,203 |
| cifar | 2 | conv | 1x1 | 8,712 | 8,452 |
| cifar | 3 | dw | 3x3 | 19,436 | 19,376 |
| cifar | 4 | conv | 1x1 | 10,888 | 10,883 |
| cifar | 5 | dw | 3x3 | 5,012 | 4,995 |
| cifar | 6 | conv | 1x1 | 5,928 | 5,923 |
| cifar | 7 | dw | 3x3 | 9,204 | 9,176 |
| cifar | 8 | conv | 1x1 | 10,536 | 10,469 |
| cifar | 9 | dw | 3x3 | 2,580 | 2,571 |
| cifar | 10 | conv | 1x1 | 7,096 | 7,077 |
| cifar | 11 | dw | 3x3 | 4,436 | 4,424 |
| cifar | 12 | conv | 1x1 | 13,240 | 13,237 |
| cifar | 15 | fc | 1x1 | 644 | 642 |

check, with +2, holds on 0. Root cause: the accepting cycle of the start write is not busy. RTL and STAMP are correct; the design's chain time with +2 includes that cycle and is the CPU-visible term, so C4b is unaffected beyond one cycle per chain call. An independent bus-level count, from the first descriptor request to the stamp request, equals the self-reported STAMP on 3 of 3 commands (6,356, 7,715, 16,616 cycles), the first such check.

## VII. Discussion and Limitations

All cycle counts come from Verilator simulation; the FPGA is used only for synthesis figures (LUT, DSP, BRAM, Fmax). No FPGA wall-clock or power measurement exists yet; the procedure is in HARDWARE_TODO.md, and wall-time speedup would be the cycle speedup scaled by the Fmax ratio of the SoC with and without the accelerator. The memory model is zero-wait on-chip RAM; an external memory would change the port-bound assumption. The accelerator covers convolution, depthwise (multiplier one) and fully connected layers only; pooling and layout ops stay in software and cap MNIST by Amdahl's law. The model has biases documented before m1, for example one extra setup cycle per pixel where a layer is requantizer bound; the setup bias proved larger than documented where the port and requantizer terms tie (above). Only two networks are

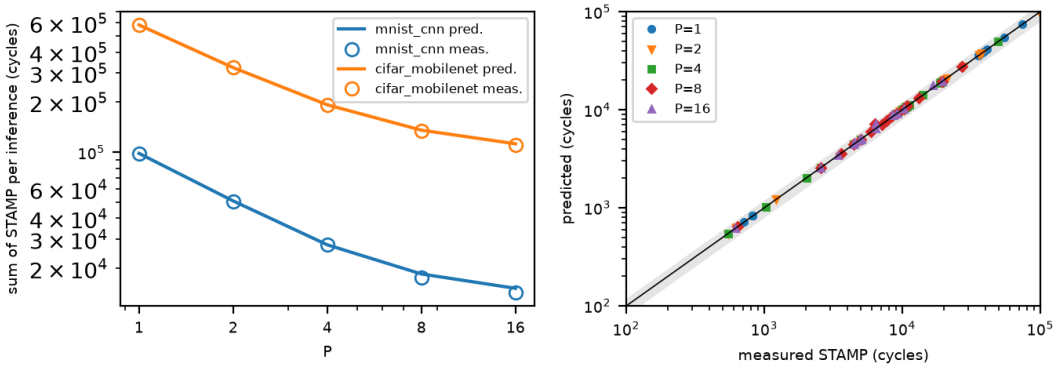



Fig. 2. Frozen prediction (tag m1, os architecture) against measurement. Left: summed STAMP per inference versus $P$ (line: prediction, circles: measurement). Right: every scored command, predicted versus measured cycles; the band is $\pm 20\,\%$.

evaluated. The repository has no remote, so the m1 hash has not been timestamped externally; this is a manual step.

## VIII. Conclusion

We built a small int8 accelerator and its backend and score an analytic model that was frozen before the RTL. The outcomes of the pre-registered claims are those in the tables above; they update from results/ through experiments/make_macros.py.

## Acknowledgment

AI assistance (Anthropic's Claude) was used to draft code and text; the authors take responsibility for all content.

## References

[1] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao, A. Ou, C. Schmidt, S. Steffl, J. Wright, I. Stoica, J. Ragan-Kelley, K. Asanovic, B. Nikolic, and Y. S. Shao, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in 58th ACM/IEEE Design Automation Conference (DAC), 2021, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1911.09925

[2] S. Prakash, T. Callahan, J. Bushagour, C. Banbury, A. V. Green, P. Warden, T. Ansell, and V. Janapa Reddi, "CFU Playground: Full-stack open-source framework for tiny machine learning (tinyML) acceleration on FPGAs," in IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2023, pp. 157–167, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2201.01863

[3] T. Moreau, T. Chen, L. Vega, J. Roesch, E. Yan, L. Zheng, J. Fromm, Z. Jiang, L. Ceze, C. Guestrin, and A. Krishnamurthy, "A hardware-software blueprint for flexible deep learning specialization," 2018, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1807.04188

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017, accessed 2026-10-10. Published at CVPR 2018 per the usual citation; the arXiv page opened lists no venue. [Online]. Available: https://arxiv.org/abs/1712.05877

[5] B. Jacob et al., "gemmlowp: fixedpoint/fixedpoint.h (saturatingroundingdoublinghighmul, roundingdividebypot)," GitHub, google/gemmlowp, accessed 2026-10-10. [Online]. Available: https://raw.githubusercontent.com/google/gemmlowp/master/fixedpoint/fixedpoint.h

[6] Google LiteRT (TensorFlow Lite) team, "Litert 8-bit quantization specification," Google AI Edge documentation, accessed 2026-10-10. [Online]. Available: https://developers.google.com/edge/litert/performance/quantization_spec

[7] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in Proceedings of the 4th MLSys Conference, 2021, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2010.08678

[8] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," 2018, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1801.06601

[9] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in Advances in Neural Information Processing Systems (NeurIPS), 2020, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2007.10319

[10] Anirudh, "mcugrad: from-scratch deep learning framework and int8 C99 compiler for microcontrollers," Git submodule third_party/mcugrad at commit e9f6283, accessed 2026-10-10. Local checkout read; the GitHub URL is the one recorded in docs/DECISIONS.md D2 and was not fetched. [Online]. Available: https://github.com/anirudh12032008/mcugrad.git

[11] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in 43rd ACM/IEEE International Symposium on Computer Architecture (ISCA), 2016, accessed 2026-10-10. [Online]. Available: https://dspace.mit.edu/handle/1721.1/102369

[12] N. P. Jouppi, C. Young, N. Patil, D. Patterson et al., "In-datacenter performance analysis of a tensor processing unit," in 44th International Symposium on Computer Architecture (ISCA), 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1704.04760

[13] NVIDIA, "NVDLA hardware architectural specification: Unit description (nvdlav1)," nvdla.org, accessed 2026-10-10. [Online]. Available: http://nvdla.org/hw/v1/ias/unit_description.html

[14] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "FINN: A framework for fast, scalable binarized neural network inference," in 25th International Symposium on Field-Programmable Gate Arrays (FPGA), 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1612.07119

[15] A. Garofalo, M. Rusci, F. Conti, D. Rossi, and L. Benini, "PULP-NN: Accelerating quantized neural networks on parallel

ultra-low-power RISC-V processors," 2019, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1908.11263

[16] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), 2015, accessed 2026-10-10. [Online]. Available: https://www.microsoft.com/en-us/research/?p=595783

[17] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2016, accessed 2026-10-10. [Online]. Available: https://compas.cs.stonybrook.edu/~mferdman/downloads.php/MICRO16_Fused_Layer_CNN_Accelerators.pdf

[18] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: An insightful visual performance model for multicore architectures," 2009, accessed 2026-10-10. Metadata from the Zenodo record; the text read was the Berkeley report version (UCB/EECS-2008-134 title: "...for Floating-Point Programs and Multicore Architectures"). [Online]. Available: https://zenodo.org/record/1236156

[19] C. Wolf, "PicoRV32: a size-optimized RISC-V cpu, readme," GitHub, YosysHQ/picorv32, accessed 2026-10-10. [Online]. Available: https://raw.githubusercontent.com/YosysHQ/picorv32/main/README.md

[20] Lattice Semiconductor, "ECP5 / ECP5-5G product page and family table," latticesemi.com, accessed 2026-10-10. [Online]. Available: https://www.latticesemi.com/en/Products/FPGAandCPLD/ECP5

[21] Anirudh, "corv32: five-stage RV32IM core and SoC (design docs, bus protocol)," Local repository /Users/anirudh/corv32, vendored subset at third_party/corv32, commit e1d0bc0, accessed 2026-10-10. No remote exists (docs/DECISIONS.md D2). [Online]. Available: file:///Users/anirudh/corv32